

Entrega 1 — Drunk Simulator

Documento que detalla cómo cumplimos cada consigna del TP, con referencias al código y a los assets relevantes.

Leyenda

-  Implementado
-  Parcialmente implementado
-  No implementado todavía

1. Pitch de venta del juego

Drunk Simulator es un juego en primera persona donde el jugador entra a un bar, elige qué tomar (cerveza, trago o whisky) y debe convivir con los efectos progresivos de la borrachera: la cámara se balancea, el movimiento se vuelve impreciso y manejar el auto se convierte en un desafío.

El jugador puede salir del bar, caminar por la ciudad, subirse al auto y volver a su casa — todo mientras el nivel de alcohol amplifica el caos. Es una sátira jugable sobre las consecuencias de tomar de más.

El pitch vive en este documento. La narrativa se sostiene en tres escenas: bar → ciudad → casa.

2. Organización de Assets y Prefabs

Toda la lógica del proyecto vive bajo Assets/Project/, separada de assets de terceros (Assets/ThirdParty/).

Assets/_Project/

├ Core/

├ Gameplay/

├ Patterns/

├ UI/

├ Audio/

└ Prefabs/

→ Managers, Input, SceneManagement, Audio

→ Player, Items, Vehicles, Systems

→ Command, EventQueue, Strategy

→ Screens, HUD, Layouts, Prefabs

→ Music, SFX, Mixers

→ Items, Environment, Player, Vehicles, Obstacles

Prefabs creados:

- Items: Cerveza.prefab, Trago.prefab, Whisky.prefab
- Environment: Banco.prefab, Calle.prefab, Casa.prefab
- Player: player.prefab

3. Tres componentes con movimiento, rotación, instanciación y/o destrucción

Cubrimos los cuatro verbos en distintos componentes:

Componente	Movimiento	Rotación	Instanciación	Destrucción
PlayerMovement.cs	 <code>controlel r.Move(...)</code>	—	—	—
MouseLook.cs	—	 <code>transfor m.localRotati on = ...</code>	—	—
PlayerPickup.cs	—	—	 <code>Instantiate(s ource)</code> (línea 265)	 <code>Destroy(currentHeldV isual)</code> (líneas 218, 252) y <code>Destroy(itemCollider)</code> (línea 276)
PickupItem.cs	—	—	—	 <code>Destroy(gameObject)</code> cuando no es <code>infiniteSup ply</code> (línea 96)
CarController.cs	 <code>rb.AddForc e(...)</code>	 <code>rb.angular rVelocity</code> para giro	—	—

4. Texturas + audio (música de fondo + ≥ 2 SFX)

Texturas: materiales en Assets/Art/Materials/ y texturas de la composición del bar en Assets/Art/Textures/BarComposition/. PickupItem.cs usa `MaterialPropertyBlock` para resaltar el ítem apuntado sin clonar materiales.

Música de fondo: BackgroundMusicManager.cs

- Se autoinicializa con `[RuntimeInitializeOnLoadMethod]` antes de cargar la primera escena (línea 23).
- Carga el clip desde Assets/Resources/Audio/BackgroundMusic.mp3.
- Persiste entre escenas con `DontDestroyOnLoad` (línea 41).

SFX (2 efectos): disparados desde PlayerPickup.cs vía `AudioSource.PlayOneShot`:

- DrinkSip.mp3 — al beber un sorbo (línea 246).
- PayDrink.mp3 — al levantar un trago de la barra (línea 227).

5. Cinemachine para el manejo de cámara ❌

No está implementado. El paquete `com.unity.cinemachine` no figura en Packages/manifest.json y ningún script lo importa.

Las cámaras actuales son manuales:

- MouseLook.cs — primera persona del jugador.
- CarFollowCamera.cs — cámara que sigue al auto con `Vector3.SmoothDamp` y `LookAt`.

Pendiente: instalar Cinemachine y reemplazar `CarFollowCamera` (y opcionalmente la cámara FPS) por una `CinemachineCamera` con un Follow + LookAt target.

6. Sistema de inputs ⚠️

Parcialmente implementado. Existe el asset `InputSystem_Actions.inputactions` con el mapeo de acciones, pero los scripts gameplay todavía usan la API vieja (`Input.GetKey`, `Input.GetAxis`):

- PlayerMovement.cs:40-41 — `Input.GetAxis("Horizontal"/"Vertical")`
- PlayerMovement.cs:55 — `Input.GetKey(KeyCode.LeftShift)` para correr
- PlayerMovement.cs:68 — `Input.GetButtonDown("Jump")`
- MouseLook.cs:37-38 — `Input.GetAxis("Mouse X"/"Mouse Y")`
- PlayerPickup.cs:94 — `Input.GetKeyDown(KeyCode.E)`
- CarController.cs:53-54 — `Input.GetAxis(...)` para acelerar/girar
- CarEnterExit.cs:24 y PlayerCarController.cs:24 — `Input.GetKeyDown(KeyCode.E)` para entrar/salir del auto.

Pendiente: generar la clase C# desde el `.inputactions` y reemplazar los `Input.X` por callbacks del nuevo Input System.

7. Colisiones / triggers con reacción ✅

Cuatro triggers que reaccionan al jugador (todos con `[RequireComponent(typeof(Collider))]` y `OnTriggerEnter`):

Trigger	Reacción	Archivo
BarDoorTrigger	Carga la escena Bar	BarDoorTrigger.cs
HomeDoorTrigger	Carga la escena City	HomeDoorTrigger.cs
BarExitTrigger	Carga City y registra el spawn BarFront	BarExitTrigger.cs
CityHomeDoorTrigger	Carga la escena Home	CityHomeDoorTrigger.cs
ParkingSpot	Detecta auto adentro y habilita salir	ParkingSpot.cs

8. Carga asincrónica de escenas

Todos los triggers de cambio de escena usan `SceneManager.LoadSceneAsync(...)`:

- BarDoorTrigger.cs:25
- HomeDoorTrigger.cs:25
- BarExitTrigger.cs:27
- CityHomeDoorTrigger.cs:25

PlayerSpawner.cs toma el `NextSpawnId` que dejó el trigger y reposiciona al jugador en el `SpawnPoint` correspondiente cuando termina la carga, manteniendo continuidad espacial entre escenas.

9. Patrones Strategy y Manager

Manager

Tres managers que centralizan estado/responsabilidad:

- DrunkManager.cs — estado de intoxicación (`alcoholLevel`, `EffectIntensity`).
- PlayerCarController.cs — coordina entrada/salida del auto y switching de cámaras.
- BackgroundMusicManager.cs — música global persistente.

Strategy

La carpeta Assets/Project/Patterns/Strategy/ existe pero está **vacía**.

Pendiente: un caso natural es el cálculo de alcohol por sorbo. Hoy `PickupItem.AlcoholPerSip` usa un `switch` sobre `PickupType`. Convertirlo a una `IDrinkStrategy` con implementaciones `BeerStrategy`, `CocktailStrategy`, `WhiskyStrategy` cumpliría la consigna.

10. Elementos de UI: scrolling (layout group) ❌

No implementado. Las carpetas Assets/Project/UI/Screens/, HUD/, Layouts/ y Prefabs/ están vacías.

Pendiente: un `ScrollRect` con `VerticalLayoutGroup` para listar tragos disponibles, créditos, o instrucciones en el menú principal.

11. Mínimo 3 eventos (actions / delegates) ⚠️

Solo 1 implementado:

- `DrunkManager.OnAlcoholLevelChanged` — `event Action<int>` disparado en cada cambio de nivel (DrunkManager.cs:19, 26, 37).

Pendiente (mínimo 2 más): candidatos naturales:

- `PlayerPickup.OnItemPickedUp` (mencionado en *CLAUDE.md* pero no presente en el código actual) — disparar al recoger un trago.
- `PlayerCarController.OnEnteredCar` / `OnExitedCar` — para que UI/audio reaccionen.
- `BarDoorTrigger.OnSceneChangeRequested` — para mostrar fade o sonido antes de cargar.

12. Escenas: menú principal, juego y cierre ⚠️

Escenas existentes: Bar.unity, City.unity, Home.unity — las tres son del flujo de juego.

Faltan:

- ❌ `MainMenu.unity` — pantalla inicial con botón "Jugar".
- ❌ `End.unity` — pantalla de victoria/derrota (p.ej. llegar a casa = victoria, chocar el auto = derrota).

Pendiente: crear ambas escenas y agregarlas al Build Settings. El cambio se puede disparar reutilizando el patrón de `BarDoorTrigger` (`SceneManager.LoadSceneAsync`).

13. Patrones Command + Event Queue ⚠️

Command ✅

Implementación completa con interfaz, comandos concretos y receptor:

- ICommand.cs — interfaz con `void Execute()`.
- EnterCarCommand.cs — encapsula `PlayerCarController.EnterCar()`.
- ExitCarCommand.cs — encapsula `PlayerCarController.ExitCar(spot)`.

Quienes encolan comandos:

- CarEnterExit.cs:29 — encola `EnterCarCommand` cuando el jugador presiona E cerca del auto.
- PlayerCarController.cs:29 — encola `ExitCarCommand` cuando se baja en un `ParkingSpot` válido.

Event Queue ⚠

La carpeta Assets/Project/Patterns/EventQueue/ está vacía, pero **CommandQueue.cs cumple la función de cola de eventos:**

- Mantiene un `Queue<ICommand>` privado (línea 6).
- Procesa **un comando por frame** en `Update()` (líneas 14-19), desacoplando productor (input) de consumidor (ejecución).

Pendiente para una implementación canónica de Event Queue: separar `IEvent` de `ICommand`, o renombrar/mover CommandQueue.cs a `Assets/_Project/Patterns/EventQueue/` con un nombre tipo `EventQueue<T>` para dejar más explícito que es el patrón Event Queue.

Resumen del estado

#	Consigna	Estado
1	Pitch	✓
2	Assets organizados + Prefabs	✓
3	3 componentes (mov / rot / instanciación / destrucción)	✓
4	Texturas + música + 2 SFX	✓
5	Cinemachine	✗
6	Sistema de inputs (nuevo)	⚠
7	Colisiones / triggers con reacción	✓
8	Carga asincrónica de escenas	✓
9	Strategy + Manager	⚠ (falta Strategy)
10	UI con layout group / scrolling	✗
11	≥3 eventos (actions / delegates)	⚠ (solo 1)
12	Escenas Menú + Juego + Cierre	⚠ (faltan menú y cierre)
13	Command + Event Queue	⚠ (Event Queue solo de facto)